



The Bug Bounty Playbook: IDOR

Part 1 of the Bug Bounty Playbook series. 252 disclosed HackerOne reports analysed. Five recurring patterns. One testing framework you can...

Abhishek meena

Published July 20, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

The attack surface map	3
Pattern 1: Sequential or predictable identifiers	4
Pattern 2: GraphQL node ID substitution	5
Pattern 3: Session-object misbinding	5
Pattern 4: Cross-tenant boundary failures	6
Pattern 5: New feature blind spots	7
The escalation ladder	7
The report that made it click	8
The testing framework	9
What is next	9
Sources	9

Part 1 of the Bug Bounty Playbook series. 252 disclosed HackerOne reports analysed. Five recurring patterns. One testing framework you can run on every target.

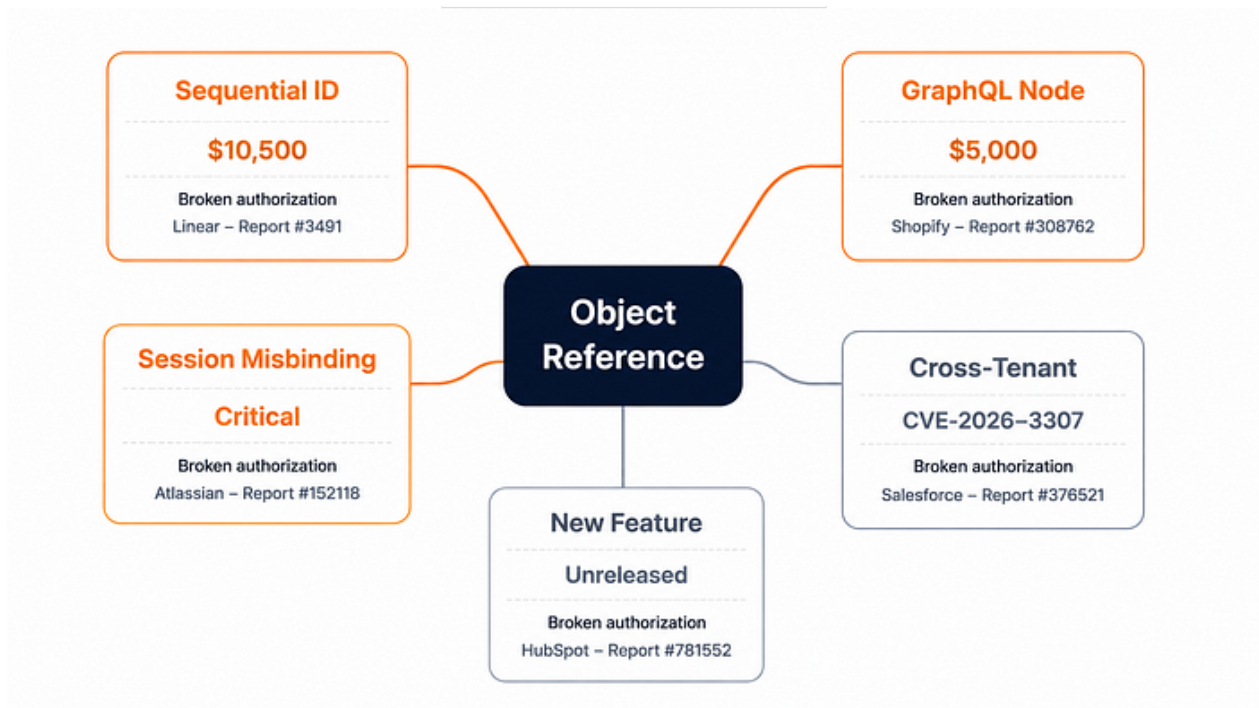
30-second version IDOR is the highest-frequency, highest-impact bug class on HackerOne. Across 252 top disclosed IDOR reports, the same five patterns repeat: sequential identifiers, GraphQL node substitution, session-object misbinding, cross-tenant boundary failures, and new-feature blind spots. The bug is never about finding the ID. It is about finding where the server forgets to check who the ID belongs to.

I have been hunting IDORs for two years. It is the bug class I would teach first if I could only teach one. Not because it is glamorous. Because it is the one bug that teaches you how to think about **authorisation**, which is the skill that separates hunters who find occasional bugs from hunters who find bugs on every target.

This Playbook is built from 252 disclosed IDOR reports on HackerOne. I ranked them by bounty and community upvotes, read the top 30 in full, and extracted the patterns that show up again and again. What follows is not a checklist. It is a mental model you can run on every endpoint you touch.

The attack surface map

Every IDOR lives at the intersection of three things: an **object reference** (an ID, a UUID, a slug), an **action** (read, update, delete), and an **authorisation check** that is either missing, wrong, or bypassable. The patterns below are the five ways that intersection breaks down in production code.



Pattern 1: Sequential or predictable identifiers

This is the most common IDOR pattern in the dataset. The server uses sequential integers or easily guessable identifiers and never checks whether the requester owns the object behind the ID.

The highest-paying IDOR on HackerOne falls here. **PayPal #415081** paid \$10,500 for a business account IDOR where an account owner could add secondary users from other accounts via the API endpoint `/businessmanage/users/api/v1/users`. The user ID was passed in the request body. The server never validated that the target user belonged to the requesting account.

The CrowdSignal report (Automattic #915114, 202 upvotes, critical) is the cleanest example in the dataset. User IDs were sequential, ranging from `00010006` to `19920500+`. The endpoint `invite-user.php?id=(userid)&popup=1` returned the victim's email when you changed the ID. Clicking "Update Permissions" logged you directly into the victim's account. **Full account takeover from a single parameter change.**

GitLab #2528293 (\$1,160, 113 upvotes) shows the same pattern in GraphQL: the ML Model Registry used incremental global IDs (`gid://gitlab/ML::Model/1000401`). Decrementing the number gave you access to private model registries across all GitLab tiers.

How to test: Find every endpoint that accepts an object identifier. Replace it with an identifier belonging to a different account (create two accounts if you have to). Watch the response. If you see data that does not belong to you, you have the bug. The test takes 30 seconds per endpoint.

Pattern 2: GraphQL node ID substitution

GraphQL makes IDOR testing systematic because every query and mutation accepts an `ID` variable. The pattern is always the same: the server resolves the node by ID but does not verify that the calling user has permission to access it.

Shopify #2207248 (\$5,000, 180 upvotes) is the textbook case. The `BillingDocumentDownload` and `BillDetails` GraphQL operations accepted a `BillingInvoice` ID. The ID was incremental and predictable. Passing another merchant's invoice ID leaked their email, full address, invoice content, and last four digits of their credit card. The researcher called it accurately: "An IDOR with numerical, predictable ID allowed anyone to dump billing details for every other shop."

HackerOne's own platform had this twice. Report #2122671 (388 upvotes) found that the `CreateOrUpdateHackerCertification` mutation accepted a certification ID with no ownership check. Changing the ID let you delete any user's certifications. Report #2218334 (208 upvotes) found the same in an unreleased feature: the `DestroyLlmConversation` mutation in the HackerOne Copilot feature accepted any conversation ID, letting you delete other users' LLM conversations before the feature was even live.

How to test: Intercept every GraphQL request in your proxy. For each one, identify the `id` variable. Replace it with an ID from a different account. The GraphQL schema tells you every operation that accepts an ID; your job is to test each one.

Pattern 3: Session-object misbinding

This is the pattern that separates intermediate hunters from advanced ones. The server has an authorisation check, but it validates the wrong thing. The session belongs to user A, but the action is applied to user B's object because the object reference comes from the request body, not the session.

Mozilla #3154983 (235 upvotes, high severity) is the most elegant example. The Firefox Accounts API had an endpoint `POST /v1/account/destroy` that accepted `email` and `authPW` in the JSON body. The server deleted the account matching the email in the body, not the account tied to the session. The attacker logged into their

own account, intercepted the destroy request, and replaced the JSON body with the victim's email and password hash. The server accepted it and deleted the victim's account.

The session was never bound to the object. The server trusted the body over the session.

Starbucks #876300 (259 upvotes, critical) is a variant: an alternate Starbucks site shared database and cookie credentials with `card.starbucks.com.sg`. A PHPSESSID from the alternate site worked on the main site. The attacker copied a session cookie from one site to another and gained access to the victim's account, including the ability to update the password and perform full takeover.

GitHub #3560256 (101 upvotes, CVE-2026-3307) is the most subtle version. The authorisation check ran against the repository in the URL, but the action was applied to a different repository specified by the `owner_id` parameter in the request body. The server checked one object but modified another.

How to test: For every state-changing endpoint, ask: does the server verify that the session owns the object in the body? Try sending your session with another user's object reference in the body. Try sending the request to endpoint A with a body that references object B. The gap between what the server checks and what it modifies is where this bug lives.

Pattern 4: Cross-tenant boundary failures

When a platform hosts multiple organisations or tenants, the authorisation check often verifies that you can access your tenant but fails to verify that the object you are manipulating belongs to the same tenant.

The GitHub report above is a cross-tenant IDOR: the check was per-repository (your repo) but the action crossed repositories. The titles in the dataset tell the same story across platforms:

- TikTok #984965: "Cross-Tenant IDOR allowing to add, update, and delete rules of any Pixel events on the platform" (84 upvotes)
- Uber #1063022: "Cross-Tenant IDOR on Business allowing escalation privilege, invitation takeover, and edition of any other Businesses' employees" (29 upvotes)
- Stripe #1066203: "GraphQL cross-tenant IDOR giving write access through the operation UpdateAtlasApplicationPerson" (25 upvotes)

How to test: Create two accounts in two different organisations or tenants. Find every endpoint that accepts a tenant-scoped object ID. Replace the ID with one from the other tenant. Cross-tenant IDORs are often rated higher because they affect every tenant on the platform.

Pattern 5: New feature blind spots

New features ship with weaker authorisation. This is not a theory. It shows up in the data.

HackerOne #2218334 (208 upvotes) was found by monitoring JavaScript files for new GraphQL queries. The researcher noticed a new `DestroyLlmConversation` mutation for the unreleased HackerOne Copilot feature. The mutation had no ownership check. The feature was not even live yet.

GitLab #2528293 targeted the ML Model Registry, introduced in GitLab 15.11 as an experiment and made generally available in 16.2. The IDOR existed because the new feature did not inherit the parent project's visibility settings.

How to test: Monitor JavaScript bundles and GraphQL schema introspection for new queries and mutations. New endpoints are the highest-ROI testing surface because they have the least review history. Every time a platform ships a feature, the authorisation model starts from scratch.

The escalation ladder

The same IDOR can be worth \$100 or \$10,500. The difference is the impact you demonstrate. Every report in the top 30 escalated beyond "I can read another user's data."

Rung 1: Read. Data disclosure. Most IDORs start here. The Nord Security report #751577 (387 upvotes) gave access to payments data of any user. No bounty was awarded, but 387 upvotes tell you the community recognised the severity. The lesson: even read-only IDORs can be high-impact if the data is sensitive.

Rung 2: Update. Modify victim data. Reddit #1661113 (206 upvotes) let an attacker modify the links of any user. The Yelp report #391092 (213 upvotes) let you order, book, buy, and reserve using another user's credit card. The escalation from read to write is what turns a medium into a high.

Rung 3: Delete. Destructive operations. HackerOne #2122671 (388 upvotes) let you delete all certifications from any user. Pornhub #380410 (\$1,500, 266 upvotes) let you delete photos and albums. Deletion IDORs almost always get rated higher because the impact is irreversible.

Rung 4: Account takeover. CrowdSignal #915114 (202 upvotes, critical) turned an IDOR into full account takeover by clicking "Update Permissions" after changing the user ID. Starbucks #876300 (259 upvotes, critical) used session sharing across sites to reach ATO. This is the rung where bounties cross \$5,000.

Rung 5: Mass exploitation. PayPal #415081 (\$10,500) affected every business account. Shopify #2207248 (\$5,000) could dump every merchant's billing data. When you can show that the ID is sequential and the vulnerability affects every user on the platform, you are at the top of the ladder.

The report that made it click

The Mozilla report (#3154983) is the one I send to hunters who say "I know what IDOR is, I just can't find them."

The researcher logged into their own Firefox account. They went to delete the account and intercepted the request to `POST /v1/account/destroy`. The body contained their own email and password hash. They cancelled the request.

Then they logged into a second account (the victim). They intercepted the same delete request and harvested the victim's email and `authPW` from the body.

Then they went back to the attacker account. They sent the destroy request with their own session but the victim's email and password hash in the body.

The server deleted the victim's account.

What makes this report exceptional is not the technique. It is the **insight**. The researcher understood that the server trusted the request body over the session. The session was irrelevant to the deletion logic. The only thing that mattered was the email and password hash in the body, and both of those can be harvested if you can log into the victim's account once.

This is the pattern that makes IDOR hunting a thinking game, not a tooling game. No scanner finds this. No automated check catches it. You find it by asking one question: **does the server verify that the session owns the object in the body?**

The testing framework

Run this on every endpoint you encounter. Five questions, 30 seconds each:

1. **Does this endpoint accept an object identifier?** If yes, proceed. Every ID, UUID, slug, and node reference is a candidate.
2. **Is the identifier sequential or predictable?** Increment it, decrement it, try a different account's ID. If you see someone else's data, you have Rung 1.
3. **Does the action go beyond read?** Test update and delete, not just read. Write operations escalate severity and bounty.
4. **Does the server bind the session to the object?** Send your session with another user's object reference in the body. If the server acts on the body object, you have a session misbinding IDOR.
5. **Is this a new feature?** Check JavaScript bundles and GraphQL schema for new operations. New features ship with weaker authorisation.

Five questions. Every endpoint. That is the framework.

What is next

This is Part 1 of the Bug Bounty Playbook series. Each part takes one bug class, mines the top disclosed reports from HackerOne, and extracts the patterns you can actually test. Next in the series: **Part 2: SSRF**, where the same methodology applied to 313 top SSRF reports reveals a completely different set of patterns: filter bypass chains, DNS rebinding tricks, and the escalation from blind SSRF to cloud metadata extraction.

If you want the framework as a checklist you can run on your next target, I have put it in a single page. Follow for the series.

Sources

1. PayPal IDOR to add secondary users (\$10,500, 786 upvotes) — hackerone.com/reports/415081 (https://hackerone.com/reports/415081)
2. HackerOne IDOR: Delete all Licenses and certifications (388 upvotes) — hackerone.com/reports/2122671 (https://hackerone.com/reports/2122671)
3. Starbucks Singapore: Account Takeover via IDOR (259 upvotes, critical) — hackerone.com/reports/876300 (https://hackerone.com/reports/876300)
4. Pornhub IDOR: delete photos and album (\$1,500, 266 upvotes) — hackerone.com/reports/380410 (https://hackerone.com/reports/380410)
5. Mozilla IDOR: Account Deletion via Session Misbinding (235 upvotes) — hackerone.com/reports/3154983 (https://hackerone.com/reports/3154983)
6. Unikrn IDOR: user enumeration and PII disclosure (\$3,000, 229 upvotes) — hackerone.com/reports/1966006 (https://hackerone.com/reports/1966006)

7. Automattic/CrowdSignal IDOR: Account Takeover (202 upvotes, critical) — hackerone.com/reports/915114 (https://hackerone.com/reports/915114)
8. HackerOne IDOR: unreleased Copilot feature (208 upvotes) — hackerone.com/reports/2218334 (https://hackerone.com/reports/2218334)
9. Shopify IDOR: GraphQL BillingDocumentDownload (\$5,000, 180 upvotes) — hackerone.com/reports/2207248 (https://hackerone.com/reports/2207248)
10. GitLab IDOR: ML Model Registry (\$1,160, 113 upvotes) — hackerone.com/reports/2528293 (https://hackerone.com/reports/2528293)
11. GitHub Cross-repository IDOR (101 upvotes, CVE-2026-3307) — hackerone.com/reports/3560256 (https://hackerone.com/reports/3560256)
12. Dataset: [reddelexc/hackerone-reports](https://github.com/reddelexc/hackerone-reports) (https://github.com/reddelexc/hackerone-reports) (252 IDOR reports curated)